Learning Heuristics for the Gapped Longest Common Subsequence Problem

Marko Djukanović[1,5], Christian Blum[2], Aleksandar Kartelj[3], Sašo Džeroski[4], and Žiga Zebec[5]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iia.csic.es
[3] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[4] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[5] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

Abstract. TODO

1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [10] is a well-known combinatorial optimization problem with numerous applications in computational biology, particularly in the analysis of molecular sequences. Given an arbitrary set of input sequences $S = \{s_1, \ldots, s_m\}$ over an alphabet Σ, the goal is to determine the longest possible subsequence common to all $s_i \in S$. In recent decades, many practically motivated variants of the LCSP have been proposed by considering additional structural or biological constraints such as the constrained, arc-preserving, repetition-free version [7, 9], among others. In this work we study the *Variable Gapped* LCSP (VGLCSP), introduced in [13] and theoretically examined in [2], and experimentally in [1]. This problem extends the standard LCSP by imposing flexible distance limits ("gaps") between consecutive symbols in the resulting subsequence, allowing these gaps to vary along the input sequences.

Formally, for each input sequence $s_i \in S$, a gap function $G_{s_i} : \{1, \ldots, |s_i|\} \mapsto \mathbb{N}$ defines the maximum distance allowed between the positions of appearances of consecutive characters for resulting common subsequence in s_i. That is, if the characters of a subsequence s appear at the positions $i_i^1 < \ldots < i_i^{|s|}$ in s_i, the gap constraint is satisfied iff $i_i^x - i_i^{x-1} \leq G_{s_i}[i_i^x] + 1$ for each $x = 2, \ldots, |s|$. A common subsequence s is considered feasible if the corresponding gap constraints hold across all $s_i \in S$. The VGLCSP represents a flexible and realistic model of sequence alignment, which is particularly useful for comparing DNA or protein sequences with variable structural distances, but also in time series analysis,

especially in settings where events are required to occur within specified temporal delays [8].

Although exact dynamic programming approaches exist for two-sequence case ($m = 2$) scenario [12], their computational complexity becomes prohibitive for larger m. To the best of our knowledge, no effective exact and only one approximate approaches for general m, a clearly NP-hard problem. The only existing approximate approach is an iterative beam search [1]. This methodology is built on the state-space problem representation and introduced space of root nodes. Each such node is explored in both ways via beam search: expanding current partial solution first by feasibly appending a sequence as its prefix, and subsequently, in another beam search call, feasibly extending that solution by concatenating letter-by-letter to its tail. At each iterations, new candidates for root nodes are detected and added in the candidate queue. Given that the methodology provides a scalable alternative to inefficient exact methods, this approach provided first results in the literature for arbitrary number of input sequences m. On the other hand, the approach relies on the use of several known hand-crafted (upper bound) heuristics for which there is an evidence from the LCS problem literature that they may offer poor guidance when m is large of Σ is small [10]. In the presence of gap constraints, it could be even worsen. The second detected research gap concerns lack of practically-motivated instances for the tackled problem as only synthetic instances are generated so far. Driven by these questions, the first gap is filled by designing a learning iterative beam search, which combines hand-crafted search guidance and the learned guidance into a superior one. To learn a search heuristic, an iterative approach is performed, able to learn the guidance in the iterative beam search approach. Shortly, the heuristic score for each node is represented as an outcome of multi-layered neural network derived in a layer-by-layer forward pass. The weights of neural network model are evolved by a genetic algorithm. The COP literature is known by utilizing similar (hybrid) methodology when learning heuristics, for example in [5] for the restricted LCS, [14] for the square LCS problem, or [16] for routing problems, to name a few. The difference between these is that their learned heuristics are not further combined with others as we did here into one ensemble method. To address the second research gap, we have derived practically-motivated problem instances by taking (two) biologically-motivated benchmark sets from LCS literature, and generating energy-aware gap constraints at each position of inherited input sequences.

Contributions. This work utilizes a framework for constructing a data-driven heuristic to guide the iterative beam search, specifically designed in the context of the *Variable Gapped Longest Common Subsequence Problem* (VGLCSP). A feed-forward neural network (FFNN) is used to parameterize the heuristic score of each node in the beam search. Instead of relying on gradient-based training, which is shown effective for the (constrained) continuous problems, the network weights are optimized using a Genetic Algorithm (GA), in particular the popular biased RKGA (BRKGA) approach. Note that here we deal with a fully non-differentiable search space when training this ML model. To establish syn-

ergy between two complementary scores, the outcome of the trained neural network and hand-crafted heuristics, a novel ensemble heuristic is introduced. This novel ensemble heuristic is clearly a new state-of-the-art guidance, according to the echausive experimental evaluation. The second contribution is the introduction of the first real-world benchmark set fo the tackled problem. In this case-scenario, gap constraints are designed to reflect on local thermodynamic stability estimated via nearest-neighbor interaction around each nucleotide. In this way, the purpose is to obtain a subsequence with stable molecular structure.

The paper is organized as follows. In Section 2, the iterative multi source beam search approach from the literature is roughly presented. Section 3 provides details on the process of learning heuristic guidance through multi-layer neural network, and the design of the combined heuristic estimator for the tacked problem. Section 8 reports a rigorous experimental analysis. Finally, Section 9 concludes the paper highlighting several future research directions.

2 Iterative Multi-Source Beam Search for GLCS Problem

The methodology of Iterative multi-source beam search (IMSBS) is proposed in [1] and here we only present it in some details. Interested readers are advised to check complete details in the above-cited paper.

This approach is derived from the general *state graph* formulation for the VGLCSP. Each node represents a partial feasible subsequence, characterized by the vector of current positions in each input sequence and the subsequence length. Specifically, a state $v = (\mathbf{p}^{L,v}, l^v)$ is induced by a partial solution s^v if: (i) $p_i^{L,v} - 1$ is the smallest index such that s^v is a subsequence of the prefix string $s_i[1] \cdots s_i[p_i^{L,v} - 1]$; (ii) $l^v = |s^v|$; and (iii) all the gaps G_{s_i} are preserved with s^v. A directed arc $\alpha = (v_1 v_2)$, labeled $lett(\alpha) = a \in \Sigma$, exists between states v_1 and v_2 if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$. To expand a state v, all letters that occur in each suffix of s_i defined by respective positions $p_i^{L,v}$ are considered. For each such letter a, the minimal positions $p_i^{L,a} \geq p_i^{L,v}$ of its appearances are identified fulfilling $p_i^{L,a} - p_i^{L,v} \leq G_{s_i}(p_i^{L,a})$. The resulting child node—if not suboptimal—is given by $w = (p^{L,a} + 1, l^v + 1)$. The root node $r := ((1, \ldots, 1), 0)$ represents the empty subsequence, while goal/target states correspond to non-extendable feasible subsequences.

Unlike the classical LCSP, the VGLCSP admits multiple (possibly exponentially many) root nodes as the starting position of a root node is constrained-free. For example, in sequences $s_1 =$ `ATGGAAAA` and $s_2 =$ `ATCCAAAA` with $G_{s_1} = G_{s_2} = 1$, the state $r = ((1, 1), 0)$ cannot reach any state with the position vector $(5, 5)$ via transitions, missing an opportunity to reach optimal subsequence `AAA` is starting the search from r. This motivates the development of IMSBS approach that works on a space of rooted nodes, which are dynamically generated over iterations and explored based on its priority. BS is a popular meta-heuristic approach [15] that performs in a breadth-first-search manner by expanding up to a certain number of nodes at the current level until the bottom is reached. This

amount of nodes is controlled by the size of beam (β)—which controls the level of greediness—and a heuristic guidance (h) for selecting them—as especially sensible algorithmic parameter.

In the IMSBS, BS is iteratively executed with a set of promising candidate root nodes as an initial beam, selected from a dynamically maintained global priority queue of candidate root nodes, controlling the diversification of the approach. Each iteration performs a complete BS on a few (num_root_nodes) top candidate root nodes, which are, beforehand, refined by the beam search with a small beam width β' and heuristic guidance h', as not all candidate nodes indeed represent root nodes. This process intensify the search and permit it from returning proven suboptimal solutions. During each BS call, new candidate root nodes are generated from the complete (non-expandable) nodes and subsequently added in the priority queue. This approach balances the intensification and diversification by several of its parameters. The stopping criteria for the IMSBS are the number of allowed beam search iterations (bs_iters), and time limit of 30 minutes.

In the remaining part of the paper, we refer to the algorithm as $\texttt{IMSBS}(\beta, h, \beta', h', num_root_nodes)$, returning an approximate solution s_{imsbs}.

3 Learning to Guide the Search

Neural networks are well-known machine learning model [6] that has shown its applicability in majority of fields [4]. There are many kind of neural networks such as the basic one, multi-layer perceptron, towards more complex Constitutional and recurrent neural networks of different architectures [11]. We will here keep the things simple, and will consider only the fully-connected (multi)layered neural networks that will serve as tool to guide the search process of our iterative beam-search approach instead of the hand-crafted versions used in [1]. For doing so, the major issue is to train the neural network, that is, setting its proper weights. In the context of classical neural networks for regression, this process is performed by alternating between forward and backward propagation, where the later is used to adapt the weights to minimize the error score on a set of training data w.r.t. a loss function. The process of weights adaptation is usually executed by employing iterative numerical procedures such as (stohastic) gradient descent [3] or versions thereof using the power of gradient in quickly reaching local optima and decreasing the overall loss w.r.t. known target values, hopping to generalize well on unseen data.

As we operate in combinatorial space of an NP-hard problem, we do not expect in general target values are known in advance, or to some extent these will be known but only for small-to-middle sized instances if an efficient exact solver exists. This limits the application of gradient descent algorithms in learning weights for such environments. Thus, we approach here differently, by employing meta-heuristic approach with a diverse population of solutions representing weights of NN, able to quickly evolve towards local optima. Therefore,

the back propagation pass in the classical NN training process is replaced by a population based meta-heuristic which adopts weights over iterations. In the feed forward pass fitness value for specific NN configuration has been determined.

Predefined Architecture of Neural Network. The heuristic score is represented by a *multi-layer perceptron* with an number of hidden layers and user-defined units per layer, the standard input and output layer. The input layer received a set of numerical values associated to subproblem to be evaluated, and the output consists of one (real) value, indicating how the considered subproblem is "important". At each layer, to catch up non-linear relations between input features, we support the use of following activation functions: `hyperbolic tangent`, `ReLU`, and `sigmoid`.

For each layer l, forward propagation is defined as:

$$\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)}),$$

where $W^{(l)}$ and $b^{(l)}$ are the weight matrix and bias vector, respectively, and ϕ denotes the activation function. All weights and biases are flattened into a **single vector**, which forms the chromosome of a GA individual.

Fitness Evaluation. TODO Given a candidate weight vector:

1. The weights are loaded into the FFNN.
2. IMSBS is executed on each training instance using the neural heuristic (guiding the search in the forward iterations as more sensible than the backward iterations).
3. The size of the best sequence obtained for each instance is recorded.

The fitness value is computed as the average solution quality over all training instances. Validation performance is computed analogously on a separate dataset and is used only for reporting.

(Q0) Speed up – evaluate fitness value by using parallelization.

3.1 Feature Extraction for the Variable-Gap LCS

To guide the search process, each partial solution (node) is represented by a fixed-length numerical feature vector capturing both the current state of the construction and the structure of the remaining search space. These features are designed to be instance-independent and comparable across problem sizes.

Let $S = \{s_1, \ldots, s_k\}$ denote the set of input sequences, and let p_i be the current matched position in sequence s_i at a given node. The vector $\mathbf{p}^L = (p_1, \ldots, p_k)$ is first normalized by the corresponding sequence lengths,

$$p_i \leftarrow \frac{p_i}{|s_i|},$$

so that progress in each sequence lies in $[0, 1]$ and becomes independent of absolute input sizes.

From this normalized position vector, four statistical descriptors are extracted: maximum, minimum, mean, and standard deviation (4 features). These quantities characterize, respectively, the most advanced and least advanced sequences, the average depth of the search, and the imbalance of progress among sequences. In addition, *the current length of the partially constructed subsequence* is included as a feature (as the fifth feature). Together, these values provide a compact description of the current search state.

To capture the feasibility of future extensions, gap-related features are computed from the positions of remaining sequences (subproblem) of input sequences. For each sequence s_i, all gap bounds associated with positions strictly after the current node position are collected. Symbolically, for each v, Gap_i^{min} is calculated as $\min\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, Gap_i^{max} is calculated as $\max\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, Gap_i^{avg} as $\mathrm{AVG}\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, and Gap_i^{std} as $\mathrm{STD}\{G_i(j) \mid j = p_i^L, i = 1, \ldots, m\}$, where $G_1, \ldots, G_m$ are gap constraints associated to $s_1, \ldots, s_m$ respectively speed up the calc. of G-stats – preprocessing... . This gives us additionally 4 features; in overall 9 features are used for the **feature configuration 1** (F1). These statistics for gap constrains summarize the remaining flexibility of the instance: large average or maximum gaps indicate greater freedom for future matches, whereas small minimum gaps and large variability typically signal tighter constraints and increased difficulty. Additional feature $|\Sigma|$, this 10 features, created the **feature configuration 2** (F2); When adding m (number of input sequences) to F2, we obtain **feature configuration 3** (F3). (Q1) uestion: s there a need for normalization of gap values of respective subproblem... (Q2) other features?

The alphabet size $|\Sigma|$ and the number of sequences (m) as the attributes provide global structural information about the instance and help the learning model adapt to varying problem scales.

Finally, all features are standardized to zero mean and unit variance prior to being passed to the learning component. The utilization of normalization is know to contribute decreased sensibility of learning process when ML models are trained. Overall, the resulting representation combines information about (i) the current construction progress, (ii) the remaining gap-constraint landscape, and (iii) global instance properties, yielding between 9 and 11 features depending on the configuration. This compact yet expressive encoding enables effective learning-based guidance of the search for high-quality variable-gap longest common subsequences.

4 Genetic Algorithm

The optimization process follows a population-based evolutionary scheme.

4.1 Initialization

Each individual is initialized by sampling weights uniformly from a predefined interval $[-w, w]$ (the value w parametrized, but fixed to 1).

4.2 Elitism

A fixed number of top-performing individuals are copied unchanged into the next generation.

4.3 Mutation

Several individuals are generated completely at random (random immigrants). This mechanism promotes exploration and maintains population diversity.

4.4 Offspring Generation

Three alternative parent-selection strategies are supported:

- **RKGA**: Two parents are selected uniformly at random, and each gene is inherited from either parent with probability 0.5.
- **BRKGA**: One parent is selected from the elite set and one from the non-elite set. Each gene is inherited from the elite parent with a predefined probability.
- **Lexicase Selection**: Training instances are randomly shuffled. Individuals are filtered iteratively by retaining those achieving the best performance on each instance. Parents are selected randomly from the remaining candidates.

We opt to use the *BRKGA* as the first option to work well.

4.5 Replacement and Termination

The new population consists of elites, mutants, and offspring. The algorithm terminates once a predefined time limit is reached (in this case we used 2 hours). The best individual encountered during training is continuously tracked and stored. (Q3) Terminate immediately when the validation value is decreased?

5 Overall Learning Procedure

The learning process alternates between evolutionary optimization of network weights and evaluation through (iterative multi-source) Beam Search. Importantly, the neural network does not directly predict solutions; instead, it learns how to *guide* the search procedure. This places the method within the class of neuro-evolutionary hyper-heuristics.

6 Algorithm

Algorithm 1 GA-Based Learning of a Beam Search Heuristic

```
Initialize population P with random neural network weights
Evaluate all individuals on training set T
Store best solution θ*
start ← current time
while time limit T_max has not exceeded do
    Sort P by fitness (descending) // Section 2.1
    Initialize empty population P'
    Copy top n_elites individuals from P to P'
    for i = 1 to n_mutants do
        Create random individual x
        Evaluate x on T
        Add x to P'
        Update θ* if improved
    end for
    for i = 1 to n_offspring do
        Select parents using RKGA, BRKGA, or Lexicase
        Generate offspring by crossover
        Evaluate offspring on T
        Add offspring to P'
        Update θ* if improved
    end for
    P ← P'
end while
return best weights θ*
```

6.1 Explanation of the GA-Based Learning Procedure

The proposed algorithm employs a Genetic Algorithm (GA) to learn the parameters of a neural network that guides a beam search heuristic. The objective is to optimize a parameter vector θ, representing the neural network weights, such that the induced heuristic achieves high performance on a given training set $T = \{I_1, I_2, \ldots, I_m\}$ of problem instances.

Initialization. The procedure starts by generating an initial population P of candidate solutions, where each individual corresponds to a randomly initialized set of neural network weights. All individuals are evaluated on the training set T, yielding a fitness value that reflects the quality of the corresponding heuristic. The best-performing individual found so far is stored as θ^*.

Fitness Function. Let $\mathcal{B}(I, \theta)$ denote the solution obtained by applying a beam search guided by parameters θ on instance I. Let $f(\cdot)$ be a problem-specific

objective function (e.g., solution quality or cost). The fitness of an individual θ is defined as:

$$F(\theta) = \frac{1}{|T|} \sum_{I \in T} f\big(\mathcal{B}(I, \theta)\big). \tag{1}$$

Depending on the problem formulation, f can represent either a maximization objective (e.g., solution value) or a minimization objective (e.g., cost), in which case an appropriate transformation is applied to ensure consistency.

Beam Search Integration. The beam search procedure maintains, at each depth level t, a set of at most k partial solutions (beam width k). Let $\mathcal{S}_t$ denote the set of candidate states at level t. For each state $s \in \mathcal{S}_t$, a feature representation $\phi(s)$ is computed and evaluated by the neural network:

$$h_\theta(s) = \mathrm{NN}_\theta(\phi(s)), \tag{2}$$

where $h_\theta(s)$ represents the heuristic score assigned to state s. The beam is then updated by selecting the top-k states according to $h_\theta(s)$:

$$\mathcal{S}_{t+1} = \mathrm{Top}_k \left\{ s' \mid s' \in \mathrm{Expand}(\mathcal{S}_t),\ h_\theta(s') \right\}. \tag{3}$$

This process continues until a terminal state is reached, yielding the final solution $\mathcal{B}(I, \theta)$.

Evolutionary Process. The algorithm iteratively improves the population until a predefined time limit $T_{\max}$ is reached. At each iteration, the population is sorted in descending order of fitness. A new population P' is constructed using three mechanisms:

- **Elitism.** A fixed number n_{elites} of top-performing individuals are directly copied from P to P', preserving the best solutions.
- **Mutation (Random Immigrants).** A set of $n_{mutants}$ individuals is generated randomly to introduce diversity. Each is evaluated using $F(\theta)$ and added to P', updating θ^* if improved.
- **Crossover (Recombination).** The remaining $n_{offspring}$ individuals are generated via recombination. Parents are selected using strategies such as RKGA, BRKGA, or Lexicase selection. Offspring inherit characteristics from both parents, are evaluated, and added to P', with updates to θ^* if necessary.

Population Update and Termination. After generating all individuals, the population is updated as $P \leftarrow P'$. The process repeats until the time limit $T_{\max}$ is reached. The algorithm returns the best solution θ^*, corresponding to the neural network weights that induce the most effective beam search heuristic on the training set.

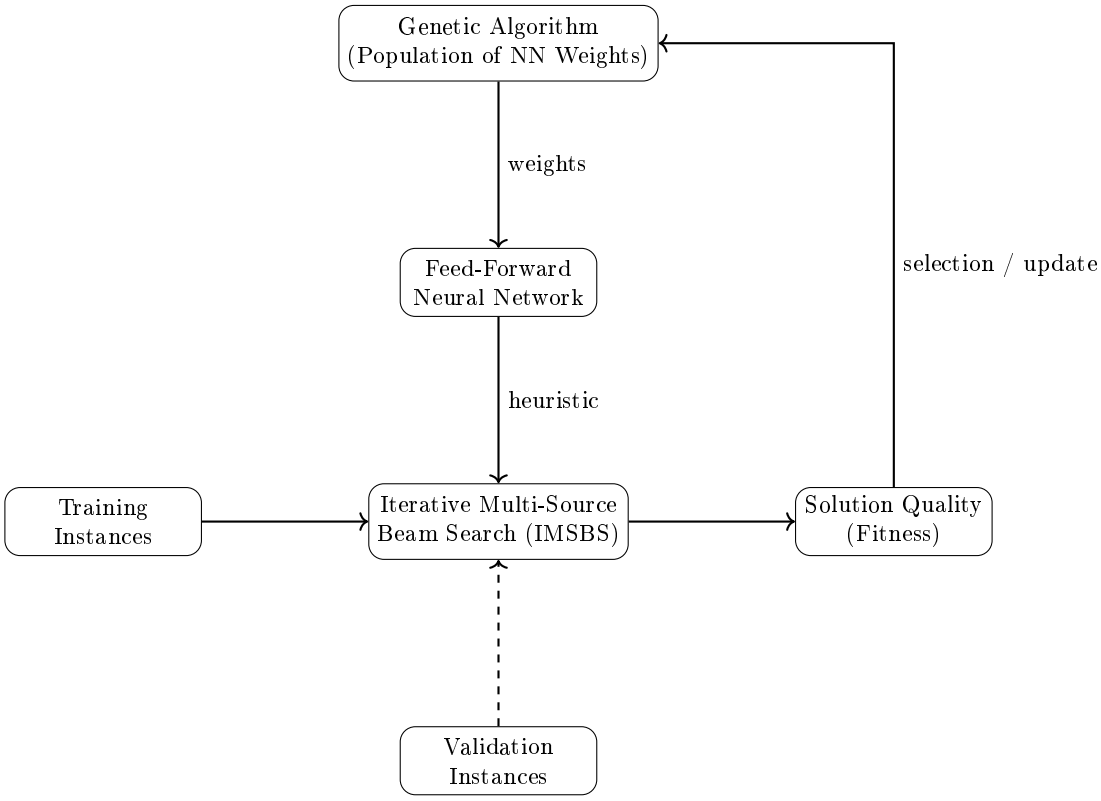

Fig. 1. Learning framework integrating a Genetic Algorithm with Iterative Multi-Source Beam Search (IMSBS). The GA evolves neural network weights. For each individual, the FFNN provides a heuristic used by IMSBS on training instances. The resulting solution quality defines fitness, which drives evolutionary selection. Validation instances are used only for monitoring generalization.

Key Properties.

- **Exploration–Exploitation Balance:** The combination of elitism, mutation, and crossover ensures both intensification and diversification.
- **Gradient-Free Optimization:** The approach does not rely on gradient information and is suitable for non-differentiable objectives.
- **Anytime Behavior:** The best solution θ^* is continuously updated and available at any point during execution.

As the best optimizer, the BRKGA setting used in training: $n_{pop} = 20, n_{elites} = 1, p_e = 0.5, p_m = 7$.

Table 1. Comparison of objective values between IMSBS and LIMSBS. Best results per instance category are in bold.

m	n	$\|\Sigma\|$	IMSBS	LIMSBS
2	50	2	**37.7**	**37.7**
2	50	4	**30.1**	29.7
2	100	2	**72.9**	72.5
2	100	4	**61.7**	60.3
2	200	2	**139.7**	139.3
2	200	4	**125.2**	120.9
2	500	2	281.9	**295.4**
2	500	4	**302.9**	285.4
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	100	2	**58.5**	**58.5**
3	100	4	**48.5**	48.0
3	200	2	**106.3**	103.4
3	200	4	**97.4**	95.6
3	500	2	133.5	**143.1**
3	500	4	**230.3**	218.8

m	n	$\|\Sigma\|$	IMSBS	LIMSBS
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	100	2	**31.5**	31.0
5	100	4	25.9	**26.9**
5	200	2	35.2	**41.6**
5	200	4	34.0	**37.2**
5	500	2	31.0	**39.3**
5	500	4	36.3	**46.2**
10	50	2	**11.4**	11.2
10	50	4	**7.1**	**7.1**
10	100	2	10.5	**12.0**
10	100	4	7.6	**8.9**
10	200	2	11.8	**12.2**
10	200	4	7.6	**8.2**
10	500	2	12.5	**12.9**
10	500	4	7.5	**8.3**

7 Discussion

The proposed framework learns heuristics rather than explicit solutions. The neural network is optimized exclusively through downstream search performance, enabling learning in non-differentiable settings. This tightly coupled integration of evolutionary learning and the classical optimization algorithm Beam Search allows the system to adapt its guidance strategy to the structure of the problem instances.

8 Experimental analysis

IMSBS setting (grid search used for tuning the algorithm):

- $\beta = 1000$, $h' = h5$, $\#imbs_iters = 1000, \#num_roots = 10$

Learning IMSBS setting uses the same setting, to be completely fair with the comparison. The neural network configuration in this case, obtained after running several 2-layer (n_1, n_2) and 3-layer configurations (n_1, n_2, n_3) for $n_1, n_2, n_3 \in \{5, 10\}$ (neurons) and $n_1 \geq n_2 \geq n_3$. We set up the same activation function in all layers choosing among the three (most) known $\{$`tanh`, `ReLU`, `sigmoid`$\}$. The best one uses `ReLU` at each layer, and besides instance-based features, it additionally incorporate the size of alphabet Σ (F2). The configurations of NNs are obtained only after 2 hours of training is a 3-layer perception with 10 neurons each.

9 Conclusions

Acknowledgments. This publication is co- funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-Curie COFUND Postdoctoral Programme grant agreement No.101081355– SMASH and by the Republic of Slovenia and the European Union from the European Regional Development Fund. Co-funded by European Union. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

References

1.
2. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
3. Amari, S.i.: Backpropagation and stochastic gradient descent method. Neurocomputing **5**(4-5), 185–196 (1993)
4. Bishop, C.M.: Neural networks and their applications. Review of scientific instruments **65**(6), 1803–1832 (1994)
5. Djukanović, M., Reixach, J., Nikolikj, A., Eftimov, T., Kartelj, A., Blum, C.: A learning search algorithm for the restricted longest common subsequence problem. Expert Systems with Applications **284**, 127731 (2025)
6. Gurney, K.: An introduction to neural networks. CRC press (2018)
7. Lafond, M., Lai, W., Liyanage, A., Zhu, B.: The longest subsequence-repeated subsequence problem. In: International Conference on Combinatorial Optimization and Applications. pp. 446–458. Springer (2023)
8. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
9. Lin, G., Chen, Z.Z., Jiang, T., Wen, J.: The longest common subsequence problem for sequences with nested arc annotations. Journal of Computer and System Sciences **65**(3), 465–480 (2002)
10. M. Djukanovic, G.R., Blum, C.: Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)
11. Mienye, I.D., Swart, T.G., Obaido, G.: Recurrent neural networks: A comprehensive review of architectures, variants, and applications. Information **15**(9), 517 (2024)
12. Peng, Y.H., Yang, C.B.: Finding the gapped longest common subsequence by incremental suffix maximum queries. Information and Computation **237**, 95–100 (2014)
13. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)
14. Reixach, J., Blum, C., Djukanović, M., Raidl, G.R.: A neural network based guidance for a brkga: An application to the longest common square subsequence problem. In: European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar). pp. 1–15. Springer (2024)
15. Wiseman, S., Rush, A.M.: Sequence-to-sequence learning as beam-search optimization. In: Proceedings of the 2016 conference on empirical methods in natural language processing. pp. 1296–1306 (2016)

16. Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. IEEE transactions on neural networks and learning systems **33**(9), 5057–5069 (2021)